Mathematical Foundations for Guidance and Navigation

Part 2. Matrix Fundamentals and Singular Value Decomposition (SVD)

Dr. Weisong Wen, Assistant Professor
Trustworthy AI and Autonomous Systems Laboratory (TAS Lab), The Hong Kong Polytechnic University
September 11, 2025

Course Overview: Matrix Fundamentals for Navigation

- **Why study matrix mathematics?**
  - Navigation algorithms rely heavily on matrix operations
  - Sensor data fusion requires matrix manipulations
  - Advanced estimation methods use matrix decompositions
  - Matrix properties determine system stability and accuracy
- **Learning objectives:**
  - Understand advanced matrix properties relevant to navigation
  - Master the Singular Value Decomposition (SVD) technique
  - Apply these concepts to navigation problems
  - Recognize when to use different matrix decompositions

Outline - Matrix Fundamentals and SVD



- Advanced Matrix Properties
  - Eigenvalues and eigenvectors
  - Positive definiteness
  - Matrix condition and stability
- Matrix Decompositions Overview
  - Types of decompositions used in navigation
  - Selecting appropriate decomposition methods
- Singular Value Decomposition (SVD)
  - Theory and intuition
  - Applications to linear systems
  - Solving navigation problems

Matrix Addition & Subtraction
For matrices of same dimensions:

$$\mathbf{A} + \mathbf{B} = [a_{ij} + b_{ij}] \tag{1}$$
$$\mathbf{A} - \mathbf{B} = [a_{ij} - b_{ij}] \tag{2}$$

Example:

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} + \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} = \begin{bmatrix} 6 & 8 \\ 10 & 12 \end{bmatrix} \tag{3}$$

Properties:

- Commutative: $\mathbf{A} + \mathbf{B} = \mathbf{B} + \mathbf{A}$
- Associative: $(\mathbf{A} + \mathbf{B}) + \mathbf{C} = \mathbf{A} + (\mathbf{B} + \mathbf{C})$
- Identity: $\mathbf{A} + \mathbf{0} = \mathbf{A}$
- Additive inverse: $\mathbf{A} + (-\mathbf{A}) = \mathbf{0}$

Navigation application:

- Combining multiple coordinate offsets
- Averaging multiple position estimates
- Combining error terms in navigation filters
- Adding displacement vectors in path planning

Matrix Multiplication
For $\mathbf{A}_{m\times n}$ and $\mathbf{B}_{n\times p}$:

$$(\mathbf{AB})_{ij} = \sum_{k=1}^{n} a_{ik} b_{kj} \tag{4}$$

Example:

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} = \tag{5}$$

$$\begin{bmatrix} 1\times5+2\times7 & 1\times6+2\times8 \\ 3\times5+4\times7 & 3\times6+4\times8 \end{bmatrix} = \tag{6}$$

$$\begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix} \tag{7}$$

Properties:

- Not commutative: $\mathbf{AB} \neq \mathbf{BA}$ (generally)
- Associative: $(\mathbf{AB})\mathbf{C} = \mathbf{A}(\mathbf{BC})$
- Distributive: $\mathbf{A}(\mathbf{B}+\mathbf{C}) = \mathbf{AB} + \mathbf{AC}$
- Identity: $\mathbf{AI} = \mathbf{IA} = \mathbf{A}$

Navigation application:

- Applying coordinate transformations
- Converting vectors between reference frames
- Sequential rotations in 3D space
- State transition in navigation algorithms

Matrix Transpose
Flips a matrix over its diagonal:

$$(\mathbf{A}^T)_{ij} = a_{ji} \tag{8}$$

Example:

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}^T = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix} \tag{9}$$

Properties:

$$(\mathbf{A}^T)^T = \mathbf{A} \tag{10}$$

$$(\mathbf{A} + \mathbf{B})^T = \mathbf{A}^T + \mathbf{B}^T \tag{11}$$

$$(\mathbf{AB})^T = \mathbf{B}^T \mathbf{A}^T \tag{12}$$

Special cases:

- **Symmetric matrix**: $\mathbf{A}^T = \mathbf{A}$
  - Example: Covariance matrices
  - $\mathbf{P} = \begin{bmatrix} \sigma_x^2 & \sigma_{xy} \\ \sigma_{xy} & \sigma_y^2 \end{bmatrix}$
- **Skew-symmetric matrix**: $\mathbf{A}^T = -\mathbf{A}$
  - Example: Cross-product operator
  - $[\vec{\omega}]_\times = \begin{bmatrix} 0 & -\omega_z & \omega_y \\ \omega_z & 0 & -\omega_x \\ -\omega_y & \omega_x & 0 \end{bmatrix}$

Navigation application:

- Converting between inertial and body coordinates
- Efficient attitude representation
- Error propagation calculations

Matrix Inverse

For square matrix $\mathbf{A}$, if it exists:

$$\mathbf{A}\mathbf{A}^{-1} = \mathbf{A}^{-1}\mathbf{A} = \mathbf{I} \tag{13}$$

Properties:

$$(\mathbf{A}^{-1})^{-1} = \mathbf{A} \tag{14}$$

$$(\mathbf{A}^T)^{-1} = (\mathbf{A}^{-1})^T \tag{15}$$

$$(\mathbf{AB})^{-1} = \mathbf{B}^{-1}\mathbf{A}^{-1} \tag{16}$$

2×2 Matrix Inverse Formula:

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix}^{-1} = \frac{1}{ad - bc} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix} \tag{17}$$

Example Calculation:

Find the inverse of $\mathbf{A} = \begin{bmatrix} 2 & 1 \\ 1 & 3 \end{bmatrix}$

Step 1: Calculate determinant

$$\det(\mathbf{A}) = 2 \times 3 - 1 \times 1 = 5 \tag{18}$$

Step 2: Apply formula

$$\mathbf{A}^{-1} = \frac{1}{5} \begin{bmatrix} 3 & -1 \\ -1 & 2 \end{bmatrix} \tag{19}$$

$$= \begin{bmatrix} 0.6 & -0.2 \\ -0.2 & 0.4 \end{bmatrix} \tag{20}$$

Navigation application:

- Solving systems of linear equations
- Reversing coordinate transformations
- Computing least-squares solutions in positioning

Determinant
For a 2×2 matrix:

$$\det \begin{bmatrix} a & b \\ c & d \end{bmatrix} = ad - bc \tag{21}$$

Example:

$$\det \begin{bmatrix} 4 & 2 \\ 1 & 3 \end{bmatrix} = 4 \times 3 - 2 \times 1 = 10 \tag{22}$$

Properties:

$$\det(\mathbf{AB}) = \det(\mathbf{A}) \det(\mathbf{B}) \tag{23}$$

$$\det(\mathbf{A}^T) = \det(\mathbf{A}) \tag{24}$$

$$\det(\mathbf{A}^{-1}) = \frac{1}{\det(\mathbf{A})} \tag{25}$$

Physical meaning in 3D transformations:

- $\det(\mathbf{A}) > 0$: Preserves orientation
- $\det(\mathbf{A}) < 0$: Reverses orientation
- $|\det(\mathbf{A})|$: Volume scaling factor
- $\det(\mathbf{A}) = 0$: Non-invertible (singular)

Examples in navigation:

- Rotation matrices: $\det(\mathbf{R}) = 1$
- Reflection matrices: $\det(\mathbf{R}) = -1$
- Scaling matrices: $\det(\mathbf{S}) = s_1 \times s_2 \times s_3$
- Used to check validity of transformation matrices

Navigation application:

- Testing if a matrix is singular before inversion

Orthogonal Matrices
A square matrix $\mathbf{Q}$ is orthogonal if:

$$\mathbf{Q}^T\mathbf{Q} = \mathbf{Q}\mathbf{Q}^T = \mathbf{I} \tag{26}$$

Key properties:

- $\mathbf{Q}^{-1} = \mathbf{Q}^T$
- $\det(\mathbf{Q}) = \pm 1$
- Preserves lengths and angles
- Columns form orthonormal basis

Example: 2D rotation matrix

$$\mathbf{R}(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \tag{27}$$

Navigation applications:

- Attitude representation
- Direction Cosine Matrices (DCM)
- Coordinate frame transformations

Symmetric Matrices
A matrix $\mathbf{S}$ is symmetric if:

$$\mathbf{S}^T = \mathbf{S} \tag{28}$$

Key properties:

- All eigenvalues are real
- Eigenvectors from different eigenvalues are orthogonal
- Can be diagonalized by orthogonal matrix

Example: Covariance matrix

$$\mathbf{P} = \begin{bmatrix} \sigma_x^2 & \sigma_{xy} \\ \sigma_{xy} & \sigma_y^2 \end{bmatrix} \tag{29}$$

Navigation applications:

- Position error representation
- Inertia tensors for spacecraft
- Kalman filter covariance matrices

Advanced Matrix Properties - I

Review from Coordinate Transformations

- Rotation matrices are orthogonal: $\mathbf{R}\mathbf{R}^T = \mathbf{I}$
- Their determinant equals 1: $\det(\mathbf{R}) = 1$
- Their inverse equals their transpose: $\mathbf{R}^{-1} = \mathbf{R}^T$

Eigenvalues and Eigenvectors

- Definition: For a square matrix $\mathbf{A}$, a non-zero vector $\mathbf{v}$ is an eigenvector with eigenvalue λ if:

$$\mathbf{A}\mathbf{v} = \lambda\mathbf{v} \tag{30}$$

- Solve by finding roots of characteristic equation:

$$\det(\mathbf{A} - \lambda\mathbf{I}) = 0 \tag{31}$$

- Physical meaning: Directions that are only scaled (not rotated) by the transformation

Key Properties

- A real $n \times n$ matrix has at most n eigenvalues
- Some may be complex numbers
- For a symmetric matrix, all eigenvalues are real
- For a rotation matrix, eigenvalues have magnitude 1

Applications in Navigation

- System stability analysis
- Principal component analysis for sensor data
- Modal analysis of dynamic systems
- Error ellipsoid visualization
- Determining principal axes of inertia
- Optimizing computational efficiency

Example 1: 2×2 Symmetric Matrix
Consider the matrix:

$$\mathbf{A} = \begin{bmatrix} 3 & 1 \\ 1 & 3 \end{bmatrix} \tag{32}$$

Step 1: Form the characteristic equation

$$\det(\mathbf{A} - \lambda \mathbf{I}) = 0 \tag{33}$$

$$\det \begin{bmatrix} 3-\lambda & 1 \\ 1 & 3-\lambda \end{bmatrix} = 0 \tag{34}$$

$$(3-\lambda)^2 - 1 = 0 \tag{35}$$

$$(3-\lambda)^2 = 1 \tag{36}$$

$$3 - \lambda = \pm 1 \tag{37}$$

Step 2: Solve for eigenvalues

$$3 - \lambda = 1 \quad \text{or} \quad 3 - \lambda = -1 \tag{38}$$

$$\lambda = 2 \quad \text{or} \quad \lambda = 4 \tag{39}$$

Step 3: Find eigenvectors for $\lambda_1 = 2$

$$(\mathbf{A} - 2\mathbf{I})\mathbf{v}_1 = \mathbf{0} \tag{40}$$

$$\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} v_{11} \\ v_{12} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \tag{41}$$

This gives us the equation $v_{11} + v_{12} = 0$, so $v_{12} = -v_{11}$

Taking $v_{11} = 1$, we get $\mathbf{v}_1 = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$

Step 4: Find eigenvectors for $\lambda_2 = 4$

$$(\mathbf{A} - 4\mathbf{I})\mathbf{v}_2 = \mathbf{0} \tag{42}$$

$$\begin{bmatrix} -1 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} v_{21} \\ v_{22} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \tag{43}$$

This gives us $-v_{21} + v_{22} = 0$, so $v_{22} = v_{21}$

Taking $v_{21} = 1$, we get $\mathbf{v}_2 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$

Verification: $\mathbf{A}\mathbf{v}_1 = 2\mathbf{v}_1$ and $\mathbf{A}\mathbf{v}_2 = 4\mathbf{v}_2$

Eigenvalue Calculation: Example 2

Example 2: Rotation Matrix
Consider a 2D rotation matrix for 90° rotation:

$$\mathbf{R} = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} \tag{44}$$

Step 1: Form the characteristic equation

$$\det(\mathbf{R} - \lambda\mathbf{I}) = 0 \tag{45}$$

$$\det \begin{bmatrix} -\lambda & -1 \\ 1 & -\lambda \end{bmatrix} = 0 \tag{46}$$

$$\lambda^2 + 1 = 0 \tag{47}$$

Step 2: Solve for eigenvalues

$$\lambda^2 = -1 \tag{48}$$

$$\lambda = \pm i \tag{49}$$

The eigenvalues are $\lambda_1 = i$ and $\lambda_2 = -i$

Step 3: Find eigenvectors for $\lambda_1 = i$

$$(\mathbf{R} - i\mathbf{I})\mathbf{v}_1 = \mathbf{0} \tag{50}$$

$$\begin{bmatrix} -i & -1 \\ 1 & -i \end{bmatrix} \begin{bmatrix} v_{11} \\ v_{12} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \tag{51}$$

This gives us $-iv_{11} - v_{12} = 0$, so $v_{12} = -iv_{11}$

Taking $v_{11} = 1$, we get $\mathbf{v}_1 = \begin{bmatrix} 1 \\ -i \end{bmatrix}$

Step 4: Find eigenvectors for $\lambda_2 = -i$

$$(\mathbf{R} + i\mathbf{I})\mathbf{v}_2 = \mathbf{0} \tag{52}$$

$$\begin{bmatrix} i & -1 \\ 1 & i \end{bmatrix} \begin{bmatrix} v_{21} \\ v_{22} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \tag{53}$$

This gives us $iv_{21} - v_{22} = 0$, so $v_{22} = iv_{21}$

Taking $v_{21} = 1$, we get $\mathbf{v}_2 = \begin{bmatrix} 1 \\ i \end{bmatrix}$

Physical meaning: A 90° rotation has no real eigenvectors!

Eigenvalue Calculation: Example 3 (Part 1)

Example 3: Navigation Application
Consider a covariance matrix from GNSS measurements:

$$\mathbf{P} = \begin{bmatrix} 25 & 15 \\ 15 & 16 \end{bmatrix} \text{ m}^2 \tag{54}$$

Step 1: Form the characteristic equation

$$\det(\mathbf{P} - \lambda \mathbf{I}) = 0 \tag{55}$$

$$\det \begin{bmatrix} 25 - \lambda & 15 \\ 15 & 16 - \lambda \end{bmatrix} = 0 \tag{56}$$

$$(25 - \lambda)(16 - \lambda) - 15^2 = 0 \tag{57}$$

$$400 - 25\lambda - 16\lambda + \lambda^2 - 225 = 0 \tag{58}$$

$$\lambda^2 - 41\lambda + 175 = 0 \tag{59}$$

Step 2: Solve for eigenvalues using the quadratic formula

$$\lambda = \frac{41 \pm \sqrt{41^2 - 4 \times 175}}{2} \tag{60}$$

$$= \frac{41 \pm \sqrt{1681 - 700}}{2} \tag{61}$$

$$= \frac{41 \pm \sqrt{981}}{2} \tag{62}$$

$$= \frac{41 \pm 31.32}{2} \tag{63}$$

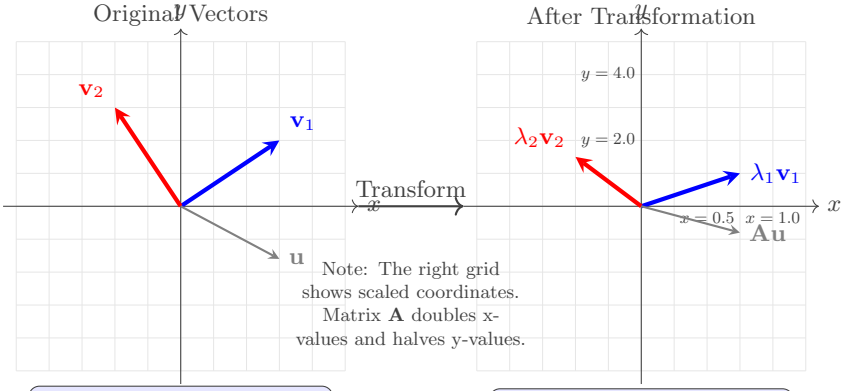
Therefore:

$$\lambda_1 \approx \frac{41 + 31.32}{2} \approx 36.1 \text{ m}^2 \tag{64}$$

$$\lambda_2 \approx \frac{41 - 31.32}{2} \approx 4.9 \text{ m}^2 \tag{65}$$

Eigenvector Equation: $\mathbf{Av} = \lambda\mathbf{v}$

Key Property: Eigenvectors maintain their direction under transformation



Note: The right grid shows scaled coordinates. Matrix **A** doubles x-values and halves y-values.

$$\mathbf{A} = \begin{bmatrix} 2 & 0 \\ 0 & 0.5 \end{bmatrix}$$

$\lambda_1 = 2$ (blue eigenvector)
$\lambda_2 = 0.5$ (red eigenvector)

Matrix from Visual Example

$$\mathbf{A} = \begin{bmatrix} 2 & 0 \\ 0 & 0.5 \end{bmatrix} \tag{66}$$

Step 1: Form the characteristic equation

$$\det(\mathbf{A} - \lambda \mathbf{I}) = 0 \tag{67}$$

$$\det \begin{bmatrix} 2-\lambda & 0 \\ 0 & 0.5-\lambda \end{bmatrix} = 0 \tag{68}$$

For a diagonal matrix, the determinant is the product of the diagonal elements:

$$(2-\lambda)(0.5-\lambda) = 0 \tag{69}$$

$$\lambda^2 - 2.5\lambda + 1 = 0 \tag{70}$$

Step 2: Solve for eigenvalues
The equation factorizes directly:

$$(2-\lambda)(0.5-\lambda) = 0 \tag{71}$$

$$\lambda = 2 \quad \text{or} \quad \lambda = 0.5 \tag{72}$$

Step 3: Find eigenvector for $\lambda_1 = 2$

$$(\mathbf{A} - 2\mathbf{I})\mathbf{v}_1 = \mathbf{0} \tag{73}$$

$$\begin{bmatrix} 0 & 0 \\ 0 & -1.5 \end{bmatrix} \begin{bmatrix} v_{11} \\ v_{12} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \tag{74}$$

This simplifies to:

$$0 \cdot v_{11} + 0 \cdot v_{12} = 0 \tag{75}$$

$$0 \cdot v_{11} + (-1.5) \cdot v_{12} = 0 \tag{76}$$

The second equation gives us: $-1.5v_{12} = 0$, which means $v_{12} = 0$
The first equation is satisfied for any value of v_{11}

Taking $v_{11} = 1$ to normalize:

$$\mathbf{v}_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \tag{77}$$

Verification for $\lambda_1 = 2$:

$$\mathbf{A}\mathbf{v}_1 = \begin{bmatrix} 2 & 0 \\ 0 & 0.5 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 2 \\ 0 \end{bmatrix} = 2 \begin{bmatrix} 1 \\ 0 \end{bmatrix} = 2\mathbf{v}_1 \tag{78}$$

Step 4: Find eigenvector for $\lambda_2 = 0.5$

$$(\mathbf{A} - 0.5\mathbf{I})\mathbf{v}_2 = \mathbf{0} \tag{79}$$

$$\begin{bmatrix} 1.5 & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} v_{21} \\ v_{22} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \tag{80}$$

This simplifies to:

$$1.5 \cdot v_{21} + 0 \cdot v_{22} = 0 \tag{81}$$

$$0 \cdot v_{21} + 0 \cdot v_{22} = 0 \tag{82}$$

The first equation gives us: $1.5v_{21} = 0$, which means $v_{21} = 0$
The second equation is satisfied for any value of v_{22}

Taking $v_{22} = 1$ to normalize:

$$\mathbf{v}_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix} \tag{83}$$

Verification for $\lambda_2 = 0.5$:

$$\mathbf{A}\mathbf{v}_2 = \begin{bmatrix} 2 & 0 \\ 0 & 0.5 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0.5 \end{bmatrix} = 0.5 \begin{bmatrix} 0 \\ 1 \end{bmatrix} = 0.5\mathbf{v}_2 \tag{84}$$

Eigenvalue Calculation: Summary and Key Insights



Results Summary:

- Eigenvalues: $\lambda_1 = 2$ and $\lambda_2 = 0.5$
- Eigenvectors: $\mathbf{v}_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$ and $\mathbf{v}_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$

Key Observations:

- For this diagonal matrix, the eigenvectors are simply the standard basis vectors
- The eigenvalues are the diagonal elements of the matrix
- Each eigenvector is stretched/compressed only along its axis:
  - $\mathbf{v}_1$ (along x-axis) is stretched by factor 2
  - $\mathbf{v}_2$ (along y-axis) is compressed by factor 0.5
- This explains the visual transformation we see in the diagram
- Any vector can be decomposed into components along these eigenvectors
- The transformation's effect is to scale each component by its corresponding eigenvalue



Vectors from Our Visual Example

In our visualization, we used:

$$\mathbf{v}_1 = \begin{bmatrix} 1.5 \\ 1 \end{bmatrix} \tag{85}$$

$$\mathbf{v}_2 = \begin{bmatrix} -1 \\ 1.5 \end{bmatrix} \tag{86}$$

$$\mathbf{u} = \begin{bmatrix} 1.5 \\ -0.8 \end{bmatrix} \tag{87}$$

Decomposition into Eigenvectors

Any vector can be written as a linear combination of eigenvectors:

$$\mathbf{v} = c_1\mathbf{e}_1 + c_2\mathbf{e}_2 \tag{88}$$

Where $\mathbf{e}_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$ and $\mathbf{e}_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$ are our calculated eigenvectors.

Expressing Our Vectors

$$\mathbf{v}_1 = 1.5\mathbf{e}_1 + 1\mathbf{e}_2 \tag{89}$$

$$\mathbf{v}_2 = -1\mathbf{e}_1 + 1.5\mathbf{e}_2 \tag{90}$$

$$\mathbf{u} = 1.5\mathbf{e}_1 - 0.8\mathbf{e}_2 \tag{91}$$

Transformation Property

For any vector $\mathbf{v} = c_1\mathbf{e}_1 + c_2\mathbf{e}_2$:

$$\mathbf{A}\mathbf{v} = \mathbf{A}(c_1\mathbf{e}_1 + c_2\mathbf{e}_2) \tag{92}$$

$$= c_1\mathbf{A}\mathbf{e}_1 + c_2\mathbf{A}\mathbf{e}_2 \tag{93}$$

$$= c_1\lambda_1\mathbf{e}_1 + c_2\lambda_2\mathbf{e}_2 \tag{94}$$

$$= 2c_1\mathbf{e}_1 + 0.5c_2\mathbf{e}_2 \tag{95}$$

Transforming Vector $\mathbf{v}_1$

$$\mathbf{A}\mathbf{v}_1 = \begin{bmatrix} 2 & 0 \\ 0 & 0.5 \end{bmatrix} \begin{bmatrix} 1.5 \\ 1 \end{bmatrix} \tag{96}$$

$$= \begin{bmatrix} 2 \times 1.5 \\ 0.5 \times 1 \end{bmatrix} \tag{97}$$

$$= \begin{bmatrix} 3 \\ 0.5 \end{bmatrix} \tag{98}$$

Alternatively, using eigenvector decomposition:

$$\mathbf{A}\mathbf{v}_1 = \mathbf{A}(1.5\mathbf{e}_1 + 1\mathbf{e}_2) \tag{99}$$

$$= 1.5(2\mathbf{e}_1) + 1(0.5\mathbf{e}_2) \tag{100}$$

$$= 3\mathbf{e}_1 + 0.5\mathbf{e}_2 \tag{101}$$

$$= \begin{bmatrix} 3 \\ 0.5 \end{bmatrix} \tag{102}$$

Transforming Vector $\mathbf{v}_2$

$$\mathbf{A}\mathbf{v}_2 = \begin{bmatrix} 2 & 0 \\ 0 & 0.5 \end{bmatrix} \begin{bmatrix} -1 \\ 1.5 \end{bmatrix} \tag{103}$$

$$= \begin{bmatrix} 2 \times (-1) \\ 0.5 \times 1.5 \end{bmatrix} \tag{104}$$

$$= \begin{bmatrix} -2 \\ 0.75 \end{bmatrix} \tag{105}$$

Transforming Vector $\mathbf{u}$

$$\mathbf{A}\mathbf{u} = \begin{bmatrix} 2 & 0 \\ 0 & 0.5 \end{bmatrix} \begin{bmatrix} 1.5 \\ -0.8 \end{bmatrix} \tag{106}$$

$$= \begin{bmatrix} 2 \times 1.5 \\ 0.5 \times (-0.8) \end{bmatrix} \tag{107}$$

$$= \begin{bmatrix} 3 \\ -0.4 \end{bmatrix} \tag{108}$$

Eigenvalues and Eigenvectors: Why They Matter in Navigation



Theoretical Importance

- Fundamental property of linear transformations
- Simplify complex matrix operations
- Enable matrix diagonalization
- Provide insight into system stability
- Characterize key directions in data

Mathematical Applications

- Principal Component Analysis (PCA)
- Solving coupled differential equations
- Optimizing computational efficiency
- Analyzing dynamic systems

Navigation Applications

- **Error Ellipses:** Semi-major and semi-minor axes of position error ellipses in GNSS
- **Inertial Navigation:** Principal axes of inertia in vehicle dynamics
- **Sensor Fusion:** Optimally combining data from multiple sensors
- **Attitude Determination:** Finding optimal rotation from measurements
- **Kalman Filtering:** Understanding error propagation in state estimation

Key Takeaway: Eigenvalues and eigenvectors provide critical insight into transformations and error characteristics, helping us design robust navigation systems.

Positive Definite Matrices

- A symmetric matrix $\mathbf{P}$ is positive definite if:

$$\mathbf{x}^T\mathbf{P}\mathbf{x} > 0 \quad \text{for all non-zero vectors } \mathbf{x} \tag{109}$$

- Equivalent conditions:
  - All eigenvalues are positive
  - All leading principal minors are positive
  - $\mathbf{P} = \mathbf{L}\mathbf{L}^T$ for some non-singular $\mathbf{L}$ (Cholesky decomposition)
- A matrix is positive semi-definite if:

$$\mathbf{x}^T\mathbf{P}\mathbf{x} \geq 0 \quad \text{for all vectors } \mathbf{x} \tag{110}$$

Navigation Applications

- Covariance matrices in estimation
  - Must be positive definite
  - Ensures physical consistency
  - Determines uncertainty ellipsoids
- Information matrices in optimization
  - Used in least squares estimation
  - Inverse of covariance matrix
- Weighting matrices in measurement
  - Used to emphasize reliable data
  - Forms quadratic cost functions

Matrix Condition Number

- Measures sensitivity to numerical errors:

$$\kappa(\mathbf{A}) = \|\mathbf{A}\| \cdot \|\mathbf{A}^{-1}\| = \frac{\sigma_{\max}}{\sigma_{\min}} \tag{111}$$

  where $\sigma_{\max}$ and $\sigma_{\min}$ are the largest and smallest singular values

- Bound on solution error:

$$\frac{\|\Delta\mathbf{x}\|}{\|\mathbf{x}\|} \leq \kappa(\mathbf{A}) \frac{\|\Delta\mathbf{b}\|}{\|\mathbf{b}\|} \tag{112}$$

  where $\mathbf{A}\mathbf{x} = \mathbf{b}$ and $\mathbf{A}(\mathbf{x} + \Delta\mathbf{x}) = \mathbf{b} + \Delta\mathbf{b}$

Interpretation

- $\kappa \approx 1$: Well-conditioned (stable)
- $\kappa \gg 1$: Ill-conditioned (unstable)
- $\kappa = \infty$: Singular (non-invertible)

Navigation Implications

- Poor satellite geometry leads to high condition numbers
  - Related to DOP (Dilution of Precision)
  - Affects accuracy of position solutions
- Guides selection of numerical methods
  - When to use regularization
  - Which decomposition to select
  - How much to trust solutions

Matrix Decompositions Overview



Common Matrix Decompositions

- **Eigendecomposition**
  - $\mathbf{A} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^{-1}$
  - For diagonalizable matrices
  - $\mathbf{Q}$ contains eigenvectors
  - $\mathbf{\Lambda}$ is diagonal with eigenvalues
  - Useful for understanding system dynamics
- **Singular Value Decomposition**
  - $\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$
  - Works for any matrix (even rectangular)
  - $\mathbf{U}$, $\mathbf{V}$ are orthogonal
  - $\mathbf{\Sigma}$ contains singular values
  - Best for solving linear systems
- **QR Decomposition**
  - $\mathbf{A} = \mathbf{QR}$
  - $\mathbf{Q}$ is orthogonal
  - $\mathbf{R}$ is upper triangular
  - Efficient for sequential processing
  - Used in real-time applications
- **Cholesky Decomposition**
  - $\mathbf{A} = \mathbf{L}\mathbf{L}^T$
  - For positive definite matrices only
  - $\mathbf{L}$ is lower triangular
  - Computationally efficient
  - Used in Kalman filtering
- **LU Decomposition**
  - $\mathbf{A} = \mathbf{LU}$
  - $\mathbf{L}$ is lower triangular
  - $\mathbf{U}$ is upper triangular
  - Efficient for solving multiple systems

Choosing the Right Decomposition

Decomposition	Best for	Restrictions	Navigation Use
Eigendecomposition	System analysis	Square matrices	Controller design
SVD	Linear systems	None	GNSS positioning
QR	Sequential processing	None	Real-time filtering
Cholesky	Efficient solving	Positive definite	Kalman filtering

Selection Criteria

- **Computational efficiency**
  - Cholesky: $\frac{1}{3}n^3$ flops
  - QR: $2n^2m - \frac{2}{3}n^3$ flops for $m \times n$ matrix
  - SVD: $4n^2m + 8nm^2 + 9m^3$ flops
- **Numerical stability** - SVD is most stable for ill-conditioned systems
- **Problem structure** - Can be exploited for efficiency
- **Required information** - Some decompositions reveal specific properties (rank, null space)



The SVD Theorem

- Any matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ can be factored as:

$$\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T \tag{113}$$

- Where:
  - $\mathbf{U} \in \mathbb{R}^{m \times m}$ is orthogonal ($\mathbf{U}^T\mathbf{U} = \mathbf{I}$)
  - $\mathbf{\Sigma} \in \mathbb{R}^{m \times n}$ is diagonal with non-negative entries
  - $\mathbf{V} \in \mathbb{R}^{n \times n}$ is orthogonal ($\mathbf{V}^T\mathbf{V} = \mathbf{I}$)
- Diagonal entries σ_i of $\mathbf{\Sigma}$ are the singular values of $\mathbf{A}$
- Singular values are arranged in descending order: $\sigma_1 \geq \sigma_2 \geq \ldots \geq \sigma_r > 0$
- Number of non-zero singular values equals the rank of $\mathbf{A}$
- The columns of $\mathbf{U}$ are called left singular vectors
- The columns of $\mathbf{V}$ are called right singular vectors

Matrix Visualization

$$\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T \tag{114}$$

Expanded form:

$$\mathbf{A} = \begin{bmatrix} | & | & \cdots & | \\ \mathbf{u}_1 & \mathbf{u}_2 & \cdots & \mathbf{u}_m \\ | & | & \cdots & | \end{bmatrix} \begin{bmatrix} \sigma_1 & 0 & \cdots & 0 \\ 0 & \sigma_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \sigma_n \end{bmatrix} \begin{bmatrix} -- & \mathbf{v}_1^T & -- \\ -- & \mathbf{v}_2^T & -- \\ & \vdots & \\ -- & \mathbf{v}_n^T & -- \end{bmatrix} \tag{115}$$

Alternative Expression

$$\mathbf{A} = \sum_{i=1}^{r} \sigma_i \mathbf{u}_i \mathbf{v}_i^T \tag{116}$$

Where r is the rank of matrix $\mathbf{A}$.

This expresses the matrix as a sum of rank-1 matrices, weighted by singular values.

Singular Value Decomposition - III: Interpretation



Geometric Interpretation

- $\mathbf{V}^T$: Rotation in input space
- $\mathbf{\Sigma}$: Scaling along new coordinate axes
- $\mathbf{U}$: Rotation in output space

Transformation Sequence

1. First, rotate the input space (using $\mathbf{V}^T$)
2. Then, scale along principal axes (using $\mathbf{\Sigma}$)
3. Finally, rotate in output space (using $\mathbf{U}$)

Key Properties

- SVD reveals principal directions and stretching factors of the transformation
- SVD always exists for any matrix (unlike eigendecomposition)
- Useful for low-rank approximations: $\mathbf{A}_k = \sum_{i=1}^{k} \sigma_i \mathbf{u}_i \mathbf{v}_i^T$
- Basis for Principal Component Analysis
- Essential for pseudo-inverse calculation: $\mathbf{A}^+ = \mathbf{V}\mathbf{\Sigma}^+\mathbf{U}^T$
- Helps solve least-squares problems
- Powerful tool for noise reduction in data

The singular values σ_i indicate the importance of each principal direction in the transformation. The largest singular values correspond to the most significant components.

Compact Form of SVD

- If $\text{rank}(\mathbf{A}) = r$, we can use reduced form:

$$\mathbf{A} = \mathbf{U}_r \mathbf{\Sigma}_r \mathbf{V}_r^T \tag{117}$$

- Where:
  - $\mathbf{U}_r \in \mathbb{R}^{m \times r}$ contains first r columns of $\mathbf{U}$
  - $\mathbf{\Sigma}_r \in \mathbb{R}^{r \times r}$ is diagonal with non-zero singular values
  - $\mathbf{V}_r \in \mathbb{R}^{n \times r}$ contains first r columns of $\mathbf{V}$
- Matrix rank = number of non-zero singular values
- Allows low-rank approximation by keeping only the largest singular values:

$$\mathbf{A}_k = \sum_{i=1}^{k} \sigma_i \mathbf{u}_i \mathbf{v}_i^T \tag{118}$$

which is the best rank-k approximation to $\mathbf{A}$ in the Frobenius norm

Advantages of Compact SVD

- Reduces storage requirements
- Improves computational efficiency
- Focuses on effective dimensions of the problem
- Reveals essential structure of the transformation
- Enables data compression

Navigation Applications

- Removing redundant measurements
- Isolating important signal components
- Reducing sensor data size
- Filtering out noise (truncated SVD)
- Computational speedups in large systems



Properties of SVD

- **Relationship to eigendecomposition**
  - Columns of $\mathbf{U}$ are eigenvectors of $\mathbf{A}\mathbf{A}^T$
  - Columns of $\mathbf{V}$ are eigenvectors of $\mathbf{A}^T\mathbf{A}$
  - σ_i^2 are eigenvalues of both $\mathbf{A}^T\mathbf{A}$ and $\mathbf{A}\mathbf{A}^T$
  - For symmetric positive definite matrices: $\sigma_i = |\lambda_i|$
- **Fundamental subspaces**
  - Range($\mathbf{A}$) is spanned by first r columns of $\mathbf{U}$
  - Null($\mathbf{A}$) is spanned by columns $r+1$ to n of $\mathbf{V}$
  - Range($\mathbf{A}^T$) is spanned by first r columns of $\mathbf{V}$
  - Null($\mathbf{A}^T$) is spanned by columns $r+1$ to m of $\mathbf{U}$

Matrix Norms via SVD

- **Frobenius norm**

$$\|\mathbf{A}\|_F = \sqrt{\sum_{i=1}^{r} \sigma_i^2} \tag{119}$$

- **2-norm** (operator norm)

$$\|\mathbf{A}\|_2 = \sigma_1 \quad \text{(largest singular value)} \tag{120}$$

- **Condition number**

$$\kappa(\mathbf{A}) = \frac{\sigma_1}{\sigma_r} \quad \text{(ratio of largest to smallest)} \tag{121}$$

- **Nuclear norm**

$$\|\mathbf{A}\|_* = \sum_{i=1}^{r} \sigma_i \quad \text{(sum of singular values)} \tag{122}$$

SVD Example Calculation - Part 1

Example: Find the SVD of matrix A

$$\mathbf{A} = \begin{bmatrix} 3 & 2 \\ 2 & 3 \end{bmatrix} \tag{123}$$

Step 1: Calculate $\mathbf{A}^T\mathbf{A}$ and $\mathbf{A}\mathbf{A}^T$

$$\mathbf{A}^T\mathbf{A} = \begin{bmatrix} 3 & 2 \\ 2 & 3 \end{bmatrix} \begin{bmatrix} 3 & 2 \\ 2 & 3 \end{bmatrix} \tag{124}$$

$$= \begin{bmatrix} 9+4 & 6+6 \\ 6+6 & 4+9 \end{bmatrix} \tag{125}$$

$$= \begin{bmatrix} 13 & 12 \\ 12 & 13 \end{bmatrix} \tag{126}$$

Similarly:

$$\mathbf{A}\mathbf{A}^T = \begin{bmatrix} 13 & 12 \\ 12 & 13 \end{bmatrix} \tag{127}$$

Step 2: Find eigenvalues of $\mathbf{A}^T\mathbf{A}$

$$\det(\mathbf{A}^T\mathbf{A} - \lambda\mathbf{I}) = 0 \tag{128}$$

$$\det \begin{bmatrix} 13-\lambda & 12 \\ 12 & 13-\lambda \end{bmatrix} = 0 \tag{129}$$

$$(13-\lambda)^2 - 12^2 = 0 \tag{130}$$

$$(13-\lambda)^2 = 144 \tag{131}$$

$$13 - \lambda = \pm 12 \tag{132}$$

So $\lambda_1 = 25$ and $\lambda_2 = 1$

Step 3: Calculate singular values

$$\sigma_i = \sqrt{\lambda_i} \tag{133}$$

Therefore:

$$\sigma_1 = \sqrt{25} = 5 \tag{134}$$

$$\sigma_2 = \sqrt{1} = 1 \tag{135}$$

Step 4: Find right singular vectors (eigenvectors of $\mathbf{A}^T\mathbf{A}$)
For $\lambda_1 = 25$:

$$(\mathbf{A}^T\mathbf{A} - 25\mathbf{I})\mathbf{v}_1 = \mathbf{0} \tag{136}$$

$$\begin{bmatrix} -12 & 12 \\ 12 & -12 \end{bmatrix} \mathbf{v}_1 = \mathbf{0} \tag{137}$$

This gives us: $-12v_{11} + 12v_{12} = 0$, so $v_{11} = v_{12}$
Normalizing: $\mathbf{v}_1 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix}$
For $\lambda_2 = 1$:

$$(\mathbf{A}^T\mathbf{A} - 1\mathbf{I})\mathbf{v}_2 = \mathbf{0} \tag{138}$$

$$\begin{bmatrix} 12 & 12 \\ 12 & 12 \end{bmatrix} \mathbf{v}_2 = \mathbf{0} \tag{139}$$

This gives us: $v_{21} = -v_{22}$
Normalizing: $\mathbf{v}_2 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \end{bmatrix}$

Step 5: Calculate left singular vectors
Method 1: Find eigenvectors of $\mathbf{A}\mathbf{A}^T$ (same as $\mathbf{A}^T\mathbf{A}$ in this case)
$\mathbf{u}_1 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix}$ and $\mathbf{u}_2 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \end{bmatrix}$
Method 2: Using the formula $\mathbf{u}_i = \frac{1}{\sigma_i}\mathbf{A}\mathbf{v}_i$
For $\mathbf{u}_1$:

$$\mathbf{u}_1 = \frac{1}{5}\mathbf{A}\mathbf{v}_1 \tag{140}$$

$$= \frac{1}{5} \begin{bmatrix} 3 & 2 \\ 2 & 3 \end{bmatrix} \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix} \tag{141}$$

$$= \frac{1}{5\sqrt{2}} \begin{bmatrix} 5 \\ 5 \end{bmatrix} \tag{142}$$

$$= \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix} \tag{143}$$

Similar calculation confirms $\mathbf{u}_2 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \end{bmatrix}$

SVD in Navigation and Coordinate Transformations - I

Coordinate Transformation Context
When $\mathbf{A}$ represents a transformation matrix:

- $\mathbf{V}$ identifies principal input directions
- $\mathbf{\Sigma}$ quantifies distortion along each direction
- $\mathbf{U}$ shows where those directions map to

Comparison with Eigendecomposition

Eigendecomposition	SVD
$\mathbf{A} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^{-1}$	$\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^{T}$
Requires square matrix	Works for any matrix
May not exist	Always exists
Same input/output space	Can connect different spaces

Rigid Transformations
For rotation matrices $\mathbf{R}$:

- All singular values $= 1$
- $\mathbf{U}$ and $\mathbf{V}$ encode rotation components
- Can detect if matrix is pure rotation
- SVD can project any matrix to closest rotation matrix

Navigation Application Examples

- **Sensor Calibration**
  - Magnetometer soft-iron corrections
  - IMU scaling and cross-axis sensitivity
  - Camera intrinsic parameter estimation

SVD in Navigation and Coordinate Transformations - II



1. GNSS Position Estimation

- DOP (Dilution of Precision) analysis:
  - Singular values of geometry matrix reveal positioning accuracy
  - Smaller values indicate higher sensitivity to errors
- Weighted least-squares solutions:
  - SVD provides stable solution even with near-singular matrices
  - SVD-based pseudoinverse: $\mathbf{A}^{+} = \mathbf{V}\boldsymbol{\Sigma}^{+}\mathbf{U}^{T}$
- Error ellipsoid orientation and size:
  - Singular vectors indicate principal error directions
  - Singular values proportional to error magnitudes

2. Map/Terrain Matching

- Finding optimal alignment between maps
- Extracting principal terrain features
- Reducing dimensionality of terrain data

3. INS Mechanization

- Stable integration of rotation matrices
- Detecting and correcting numerical errors
- Analysis of error propagation in inertial solutions
- Correcting accumulated drift in attitude matrices

Key Course Connection

- SVD provides mathematical tools to understand linear transformations in navigation
- Essential for robust implementation of navigation algorithms
- Bridges mathematical theory with practical sensor processing

Solving Linear Systems with SVD

- For system $\mathbf{Ax} = \mathbf{b}$, we can use SVD of $\mathbf{A} = \mathbf{U\Sigma V}^T$:

$$\mathbf{U\Sigma V}^T\mathbf{x} = \mathbf{b} \tag{144}$$

$$\mathbf{\Sigma V}^T\mathbf{x} = \mathbf{U}^T\mathbf{b} \tag{145}$$

$$\mathbf{V}^T\mathbf{x} = \mathbf{\Sigma}^+\mathbf{U}^T\mathbf{b} \tag{146}$$

$$\mathbf{x} = \mathbf{V\Sigma}^+\mathbf{U}^T\mathbf{b} \tag{147}$$

- Pseudoinverse $\mathbf{\Sigma}^+$ created by:
  - Taking reciprocal of non-zero singular values
  - Setting 0 entries to remain 0
  - Transposing the resulting matrix
- Moore-Penrose pseudoinverse:

$$\mathbf{A}^+ = \mathbf{V\Sigma}^+\mathbf{U}^T \tag{148}$$

Properties of Pseudoinverse Solution

- For overdetermined systems ($m > n$):
  - Minimizes residual $\|\mathbf{Ax} - \mathbf{b}\|_2$
  - Gives least squares solution
  - More stable than normal equations
  - $(\mathbf{A}^T\mathbf{A})^{-1}\mathbf{A}^T$
- For underdetermined systems ($m < n$):
  - Gives minimum norm solution
  - $\|\mathbf{x}\|_2$ is minimized among all solutions
  - Often desirable in navigation to avoid large controls or estimates
- For exactly determined systems ($m = n$):
  - If $\mathbf{A}$ is full rank, gives unique solution
  - Otherwise, gives minimum norm least squares solution

SVD for Linear Systems - II

Handling Ill-Conditioned Systems

- For small singular values, reciprocals become very large
- Can lead to numerical instability when $\sigma_i \approx 0$
- Solutions:
  - **Truncated SVD:** Set all $\sigma_i <$ threshold to zero
  - **Tikhonov regularization:** Replace σ_i^{-1} with $\frac{\sigma_i}{\sigma_i^2+\alpha^2}$
- Regularized solution:

$$\mathbf{x}_\alpha = \sum_{i=1}^{r} \frac{\sigma_i}{\sigma_i^2 + \alpha^2}(\mathbf{u}_i^T \mathbf{b})\mathbf{v}_i \tag{149}$$

where α is the regularization parameter

Navigation Applications

- **GNSS Positioning**
  - Poor satellite geometry can create ill-conditioned systems
  - Regularization improves solution stability
  - Truncated SVD can remove weak satellite contributions
  - Reveals which measurements contribute most to solution
- **Sensor Calibration**
  - Determining sensor biases and scale factors
  - Handling coplanar or near-coplanar sensor arrangements
  - Resolving ambiguities in calibration data
- **Estimation Problems**
  - State estimation with redundant or deficient measurements
  - Handling near-singular estimation problems

Simple Example
Consider the matrix:

$$\mathbf{A} = \begin{pmatrix} 4 & 0 \\ 3 & -5 \end{pmatrix} \tag{150}$$

Steps:

1. Compute $\mathbf{A}^T\mathbf{A}$:

$$\mathbf{A}^T\mathbf{A} = \begin{pmatrix} 4 & 3 \\ 0 & -5 \end{pmatrix} \begin{pmatrix} 4 & 0 \\ 3 & -5 \end{pmatrix} \tag{151}$$

$$= \begin{pmatrix} 25 & -15 \\ -15 & 25 \end{pmatrix} \tag{152}$$

2. Find eigenvalues by solving $\det(\mathbf{A}^T\mathbf{A} - \lambda\mathbf{I}) = 0$:

$$\lambda_1 = 40, \lambda_2 = 10 \tag{153}$$

3. Singular values are square roots of eigenvalues:

4. Find eigenvectors of $\mathbf{A}^T\mathbf{A}$ (columns of $\mathbf{V}$):

$$\mathbf{V} = \begin{pmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ -\frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \end{pmatrix} \tag{155}$$

5. Compute $\mathbf{U}$ columns using:

$$\mathbf{u}_i = \frac{1}{\sigma_i}\mathbf{A}\mathbf{v}_i \tag{156}$$

Computing $\mathbf{u}_1$ and $\mathbf{u}_2$ explicitly:

$$\mathbf{u}_1 = \frac{1}{6.32}\mathbf{A}\mathbf{v}_1 = \begin{pmatrix} 0.449 \\ 0.894 \end{pmatrix} \tag{157}$$

$$\mathbf{u}_2 = \frac{1}{3.16}\mathbf{A}\mathbf{v}_2 = \begin{pmatrix} -0.894 \\ 0.449 \end{pmatrix} \tag{158}$$

6. Final SVD:

$$\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T \tag{159}$$

where $\mathbf{\Sigma} = \text{diag}(6.32, 3.16)$

Quick Quiz: SVD Properties

Take a moment to check your understanding:

1. Which of these is TRUE about the singular values of a matrix?
   A) They are always positive or zero
   B) They are the square roots of eigenvalues of $\mathbf{A}$
   C) Their number equals the smaller dimension of the matrix
   D) They are always the same as the eigenvalues

2. The condition number of a matrix can tell us:
   A) The exact error in numerical solutions
   B) How sensitive solutions are to changes in input data
   C) The rank of the matrix precisely
   D) Whether the matrix is symmetric or not

3. What makes SVD particularly valuable for navigation problems?
   A) It always produces a unique solution
   B) It works for any matrix shape and reveals rank deficiencies
   C) It is the fastest decomposition method
   D) It requires the least memory

Quick Quiz: SVD Properties - Solutions

1. Which of these is TRUE about the singular values of a matrix?
   Answer: A) They are always positive or zero
   - Singular values are, by definition, non-negative
   - They are square roots of eigenvalues of $\mathbf{A}^T\mathbf{A}$, not of $\mathbf{A}$
   - The number of non-zero singular values equals the rank of the matrix
   - For non-symmetric matrices, singular values differ from eigenvalue magnitudes

2. The condition number of a matrix can tell us:
   Answer: B) How sensitive solutions are to changes in input data
   - Condition number κ measures how errors in input propagate to output
   - Relative error in output can be up to κ times relative error in input
   - High condition number indicates an ill-conditioned problem
   - Small changes in $\mathbf{b}$ can cause large changes in the solution $\mathbf{x}$

3. What makes SVD particularly valuable for navigation problems?
   Answer: B) It works for any matrix shape and reveals rank deficiencies
   - SVD applies to any rectangular matrix, unlike eigendecomposition
   - It immediately reveals rank deficiency through zero singular values
   - In GNSS positioning, rank deficiency indicates insufficient satellite geometry
   - It provides a solution that minimizes error even in underdetermined systems
   - SVD handles both overdetermined systems (more measurements than unknowns) and underdetermined systems (fewer measurements than unknowns)

SVD is the most general and robust matrix decomposition, making it ideal for complex navigation problems with varying geometry and measurement availability.

Summary and Key Takeaways



Advanced Matrix Properties

- Eigenvalues reveal system characteristics
- Positive definiteness ensures stability
- Condition number predicts numerical behavior
- Matrix decompositions offer different advantages

Matrix Decompositions

- Different decompositions for different problems
- Selection based on problem structure
- Navigation applications have specific requirements
- Computational efficiency vs. numerical stability

Singular Value Decomposition

- Universal decomposition for any matrix
- Reveals key matrix properties (rank, condition)
- Provides stable solutions to linear systems
- Handles rank deficiency naturally
- Offers regularization methods for ill-conditioned problems

Navigation Applications

- GNSS position estimation
- Sensor calibration
- State estimation
- Data compression
- System analysis
- Filtering algorithms

Matrix Calculation Tools and Solvers

Software Libraries
General-purpose mathematical:

- **MATLAB/Octave**
  - Built-in matrix operations
  - `eig`, `svd`, `inv`, `pinv`
  - Optimized for matrix manipulations
- **Python Ecosystem**
  - `NumPy`: `np.linalg` module
  - `SciPy`: Advanced decompositions
  - `SymPy`: Symbolic matrix calculations
- **Specialized Libraries**
  - `LAPACK/BLAS`: Industry standard
  - `Eigen` (C++): Template library
  - `Armadillo`, `Blaze`, `Eigen3`

Navigation-Specific Tools
Navigation libraries:

- **RTKLIB**: GNSS positioning algorithms
- **GeographicLib**: Geodetic transformations
- **GPSTK**: GPS toolkit with matrix utilities
- **ROS tf2**: Transformation framework

Implementation Considerations:

- **Embedded systems**:
  - Fixed-point vs. floating-point
  - Memory constraints
  - Real-time requirements
- **Numerical stability**:
  - Avoid direct inversion when possible
  - Use decompositions (LU, QR, SVD)
  - Consider condition number
  - Regularization techniques

Matrix Computation Tutorial - Part 1: Introduction



Tutorial Overview

Objective: Compare computational accuracy and efficiency of matrix operations between C++/Eigen and Python/NumPy

Learning Outcomes:

- Understand performance differences between C++ and Python for matrix operations
- Identify accuracy trade-offs in different numerical implementations
- Learn to properly benchmark numerical algorithms
- Apply findings to navigation algorithm implementation decisions

Setup Instructions:

1. Download starter code from course website
2. Install required libraries:
   - C++: Eigen, Chrono
   - Python: NumPy, SciPy, time, memory$_{p}rofiler$
3. Follow README instructions to compile C++ code
4. Use provided scripts to run comparison tests



Operations to Implement:

- Matrix multiplication
  - Standard matrix product
  - Element-wise multiplication
- Matrix inversion
  - Direct inversion
  - Using decompositions
- Eigenvalue decomposition
  - Full decomposition
  - Only eigenvalues
- Singular Value Decomposition
  - Full vs. reduced SVD
- Least squares solution

Testing Parameters:

Matrix dimensions:

- Small: 10×10
- Medium: 100×100
- Large: 1000×1000

Data types:

- Single precision (float)
- Double precision (double)

Matrix types:

- Random matrices
- Nearly singular matrices
- Structured matrices (e.g., diagonal dominant)
- Orthogonal matrices

Number of iterations:

- 10, 100, 1000 (operation dependent)



Metrics to Measure:

- **Execution time**
  - Wall clock time
  - CPU time
- **Memory usage**
  - Peak memory
  - Memory growth
- **Numerical accuracy**
  - Frobenius norm error
  - Maximum element-wise error
  - Relative error
- **Scaling behavior**
  - Time vs. matrix size
  - Memory vs. matrix size

Accuracy Comparison:

- Use higher precision library for reference solutions
- Test with well-conditioned and ill-conditioned matrices
- Verify correctness of operations ($A \times A^{1} = I$)

Performance Analysis:

- Use consistent hardware for all tests
- Disable background processes during benchmarking
- Run multiple iterations and report statistics
- For C++: test with different compiler optimizations
- For Python: compare NumPy with native implementation



Sample Code Snippets

C++/Eigen:

```
int main()  // Create random matrix Eigen::MatrixXd
A = Eigen::MatrixXd::Random(100, 100);
// Time SVD decomposition auto start =
std::chrono::high_resolution_clock :: now(); Eigen ::
JacobiSVD < Eigen :: MatrixXd >
svd(A); autoend = std :: chrono ::
high_resolution_clock :: now();
std::chrono::duration<double> elapsed = end-start;
std::cout << "SVD Time: " << elapsed.count() << "s"
<< std::endl;
return 0;
```

Python/NumPy:

```
Create random matrix A = np.random.rand(100, 100)
Time SVD decomposition start = time.time() u, s, vh
= np.linalg.svd(A) end = time.time()
print(f"SVD Time: end-start:.6fs")
```

Memory usage can be tracked with:

```
@memory_profiler.profiledefrun_svd(A) :
returnnp.linalg.svd(A)
```

Questions?

Dr. Weisong Wen
Assistant Professor

Trustworthy AI and Autonomous Systems Laboratory (TAS Lab)
Department of Aeronautical and Aviation Engineering
The Hong Kong Polytechnic University

Email: welson.wen@polyu.edu.hk
Office: R820 Phone: 3400 8234